

Parallel & Distributed Computing



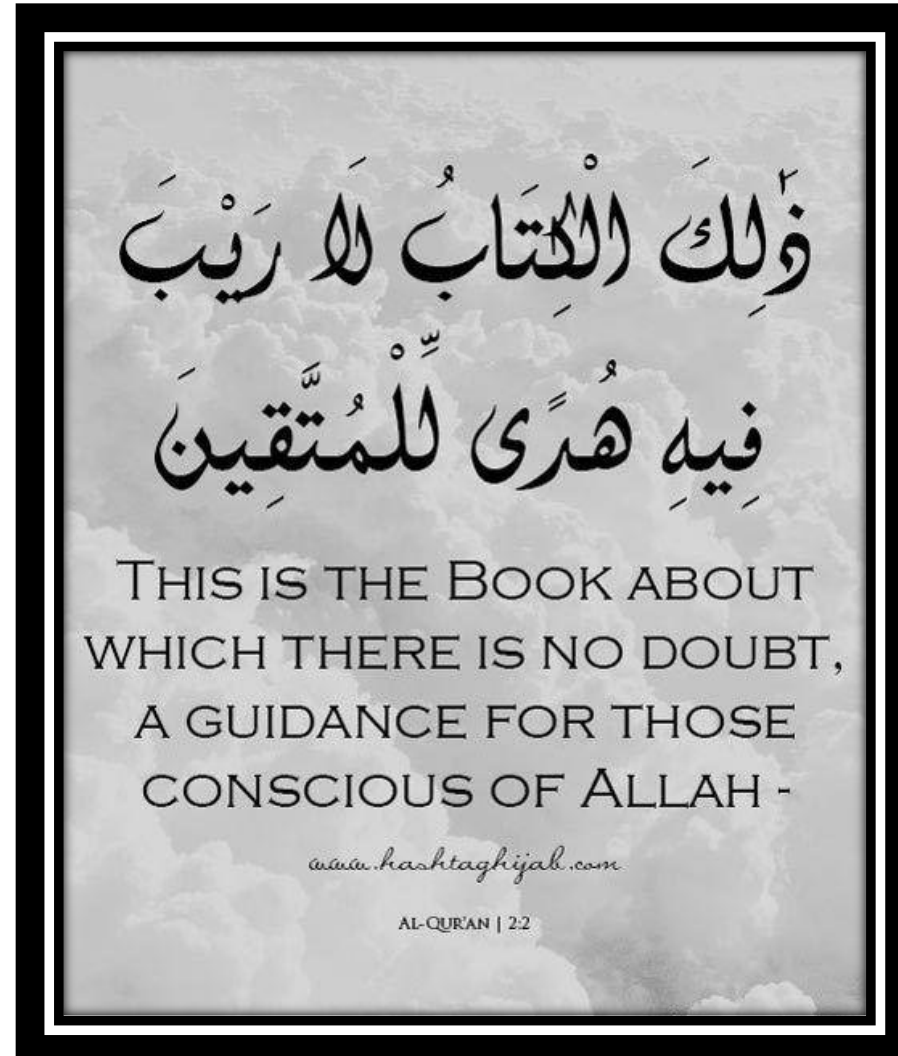
HARDWARE ARCHITECTURES & SHARED MEMORY



Dr. Muzamil Ahmed (Assistant Professor)
Namal University Mianwali

Lesson from Quran

2



Lecture Outline

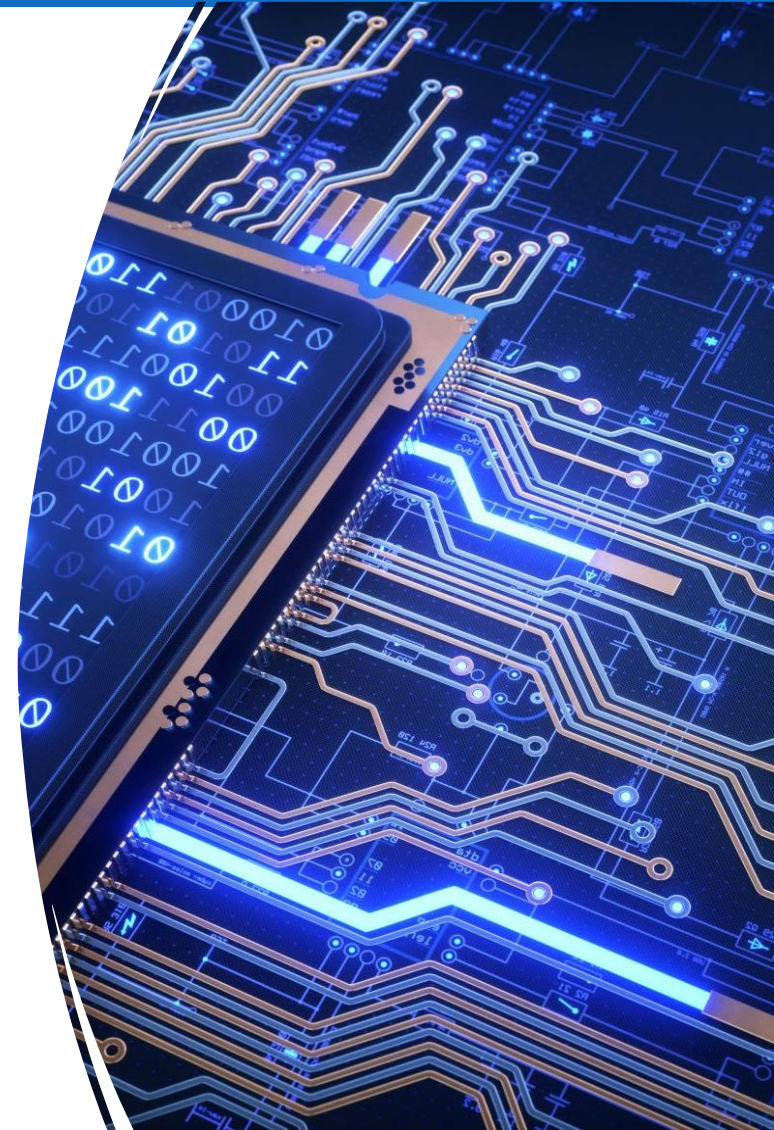
3

- ❑ Hardware Architectures
- ❑ Flynn's Taxonomy
- ❑ Single instruction stream, single data stream (SISD)
- ❑ Single instruction stream, multiple data streams (SIMD)
- ❑ Multiple instruction streams, single data stream (MISD)
- ❑ Multiple instruction streams, multiple data streams (MIMD)
- ❑ Shared Memory
- ❑ Shared Memory (Software Level + Hardware Level)



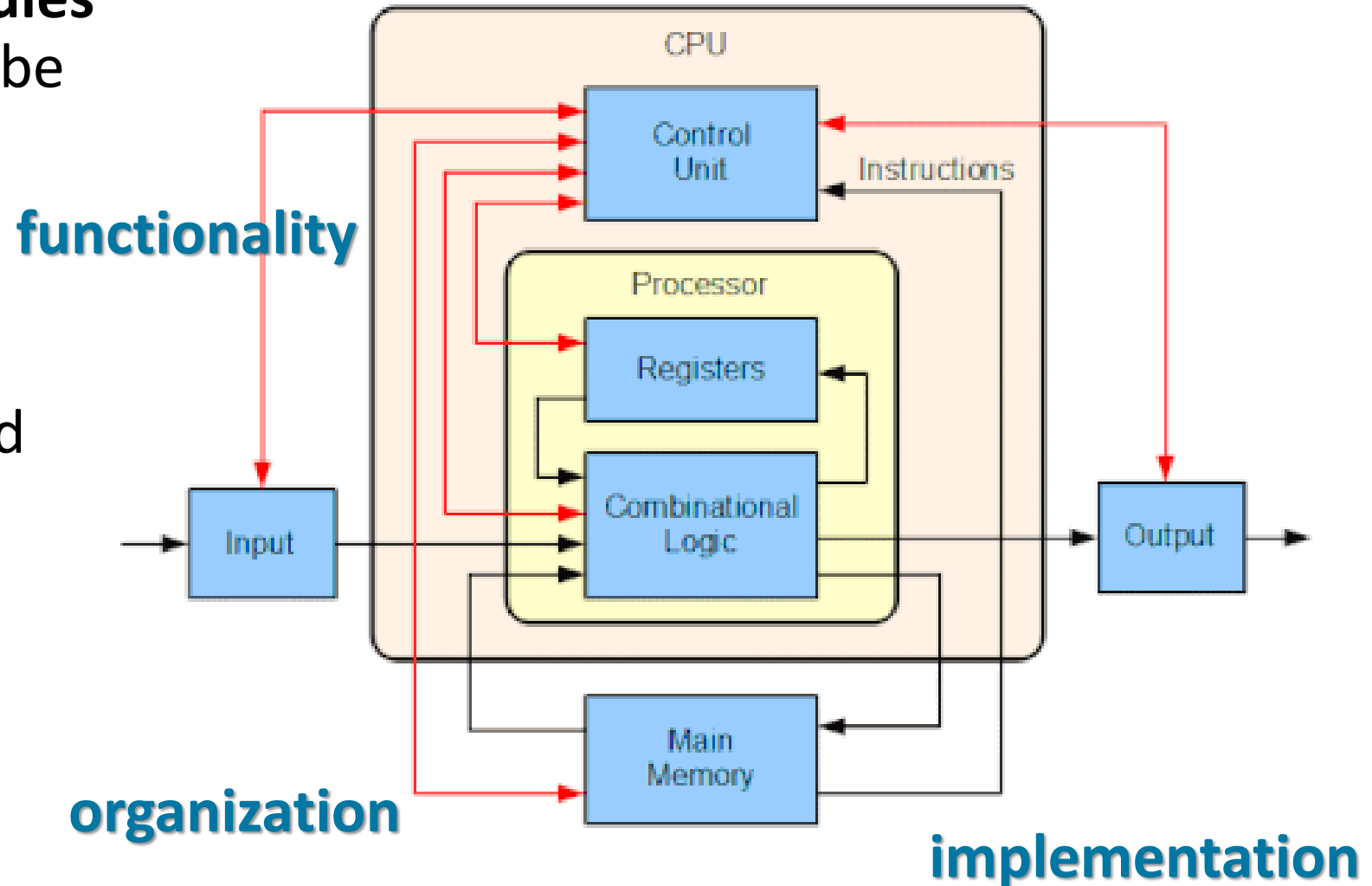
Hardware Architectures

- **Hardware architecture** refers to the identification of a **system's physical components and their interrelationships**.
- This description, often called a **hardware design model**, allows hardware designers to understand how *their components fit into a system architecture* and provides to software component designers *important information* needed for *software development and integration*.



Hardware Architectures

- **Architecture** is a set of **rules and methods** that describe the **functionality, organization, and implementation** of computer.
- Architecture is concerned with **balancing the performance, efficiency, cost, and reliability** of a computer system.



Hardware Architectures

Flynn's Taxonomy

- **Michael J. Flynn** (born May 20, 1934) is an American professor at Stanford University.
- Flynn proposed the **Flynn's taxonomy**, a **method of classifying digital computer architectures**, in 1966



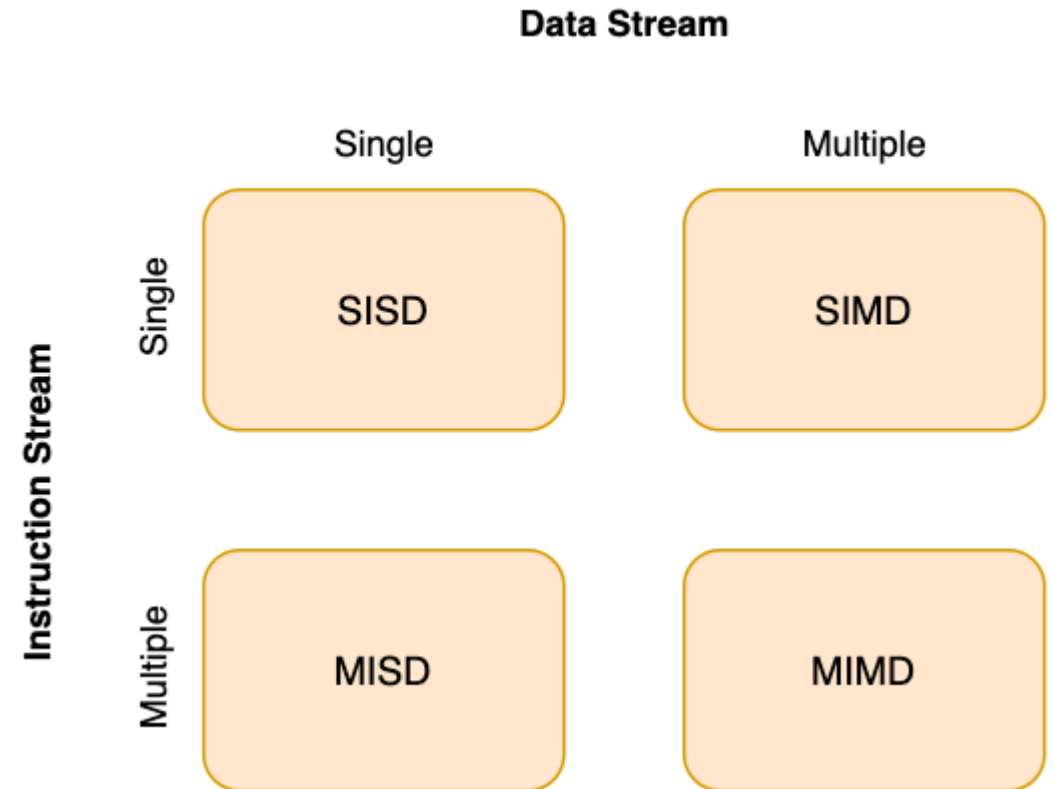
Flynn's Taxonomy

The four classifications based upon the number of **concurrent instruction (or control) streams** and **data streams** available in the architecture:

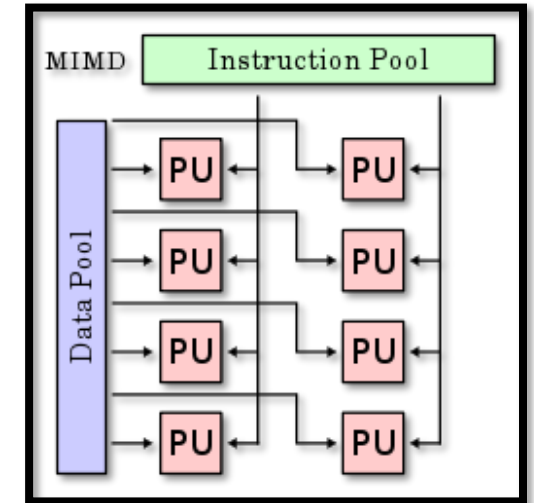
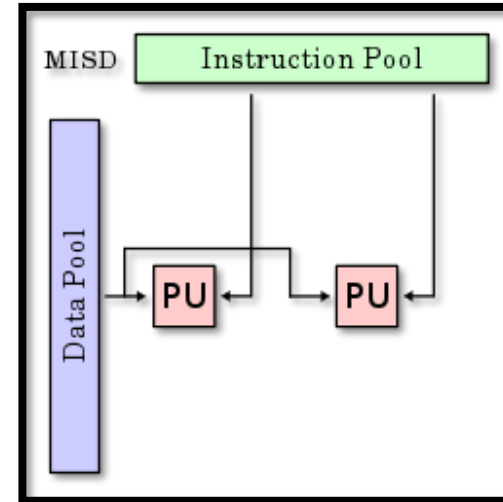
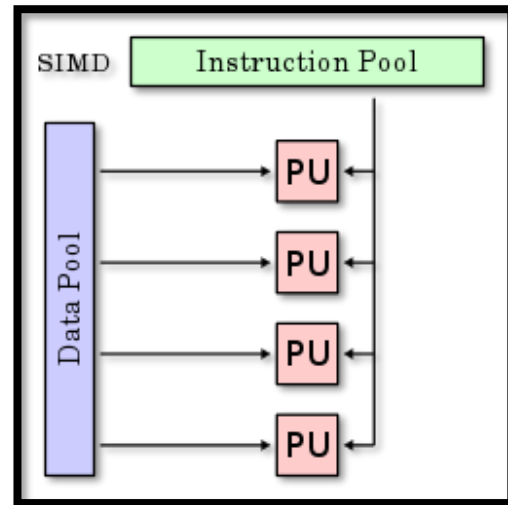
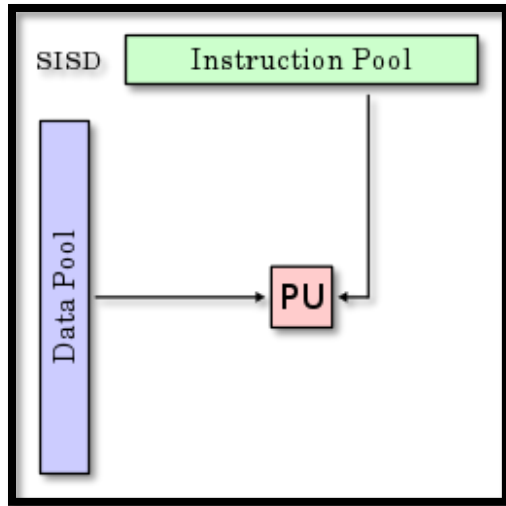
1. **Single instruction stream, single data stream (SISD)**
2. **Single instruction stream, multiple data streams (SIMD)**
3. **Multiple instruction streams, single data stream (MISD)**
4. **Multiple instruction streams, multiple data streams (MIMD)**

Flynn's Taxonomy

- **Instruction stream:** The sequence of **instructions** from **memory to the control unit**.
- **Data stream:** The sequence of **data** from **memory to the control unit**.

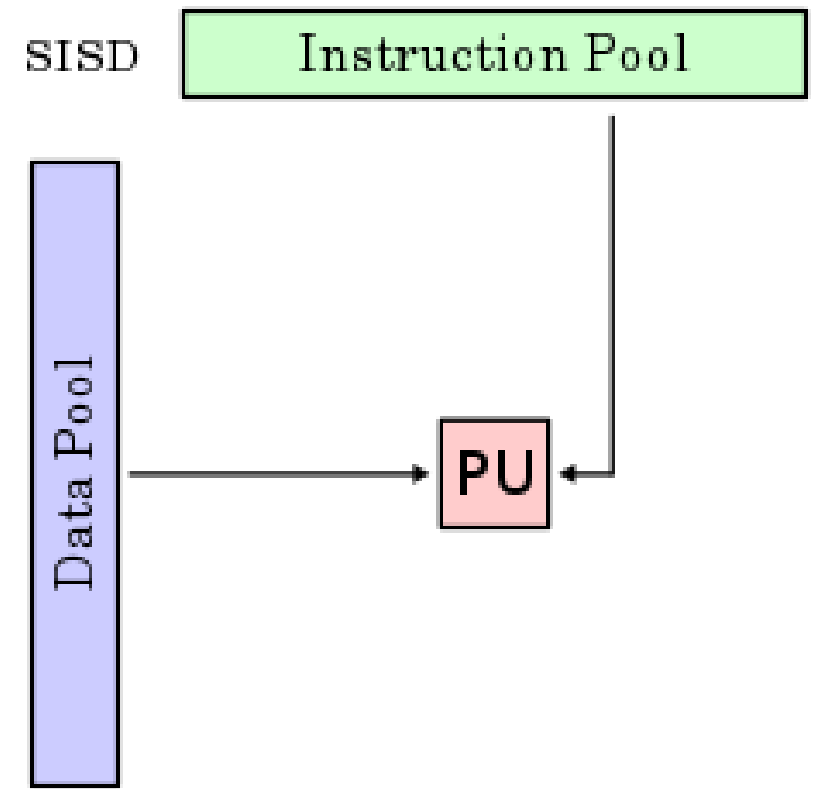


Hardware Architectures cont...



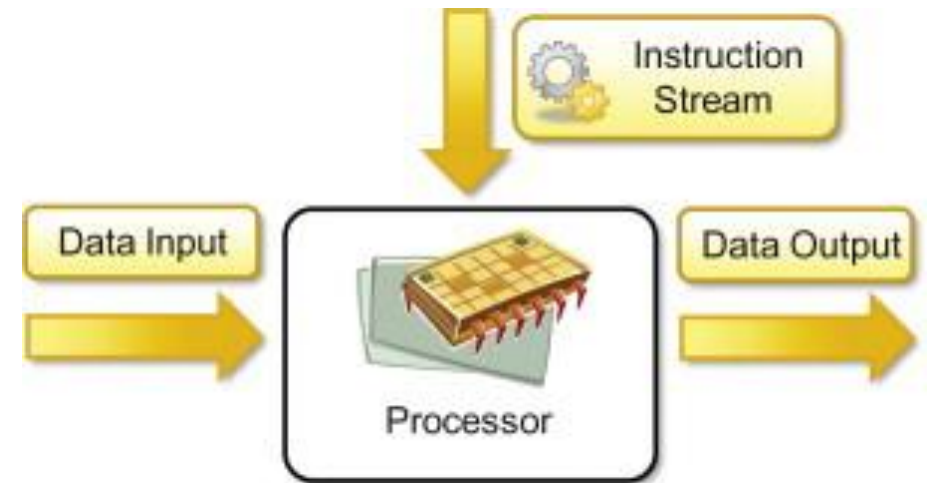
Single instruction stream, single data stream (SISD)

- A sequential computer which exploits **no parallelism** in either the **instruction** or **data streams**.
- **Single control unit** (CU) fetches **single instruction stream** (IS) from memory.
- The **CU** then generates appropriate control signals to direct single processing element (PE) to operate on single data stream (DS) i.e., one operation at a time.



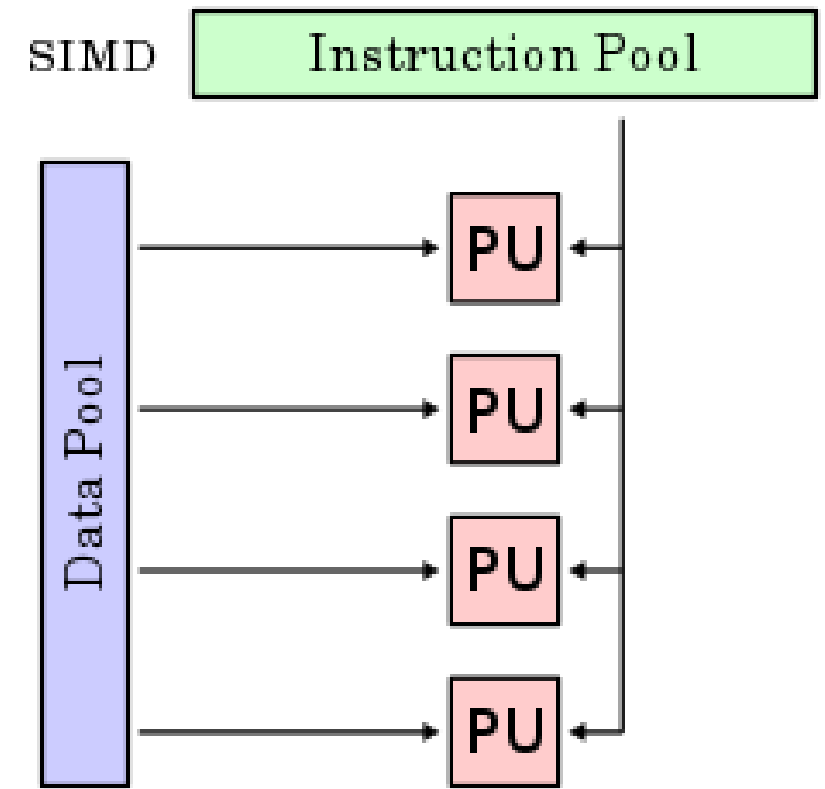
Single instruction stream, single data stream (SISD)

- ❑ It requires less power.
- ❑ There is no issue of complex communication protocol between multiple cores.
- ❑ The speed of SISD architecture is limited just like single-core processors.
- ❑ It is not suitable for larger applications.



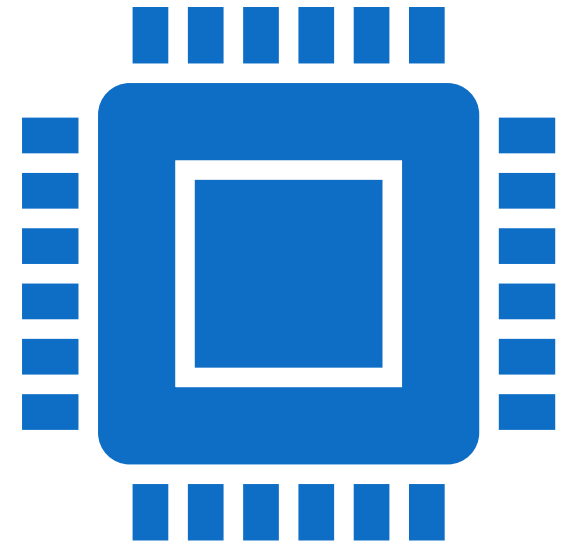
Single instruction stream, multiple data streams

- A **single instruction** operates on **multiple different data streams**.
- Instructions can be executed **sequentially**, such as by **pipelining**, or in parallel by **multiple functional units**.



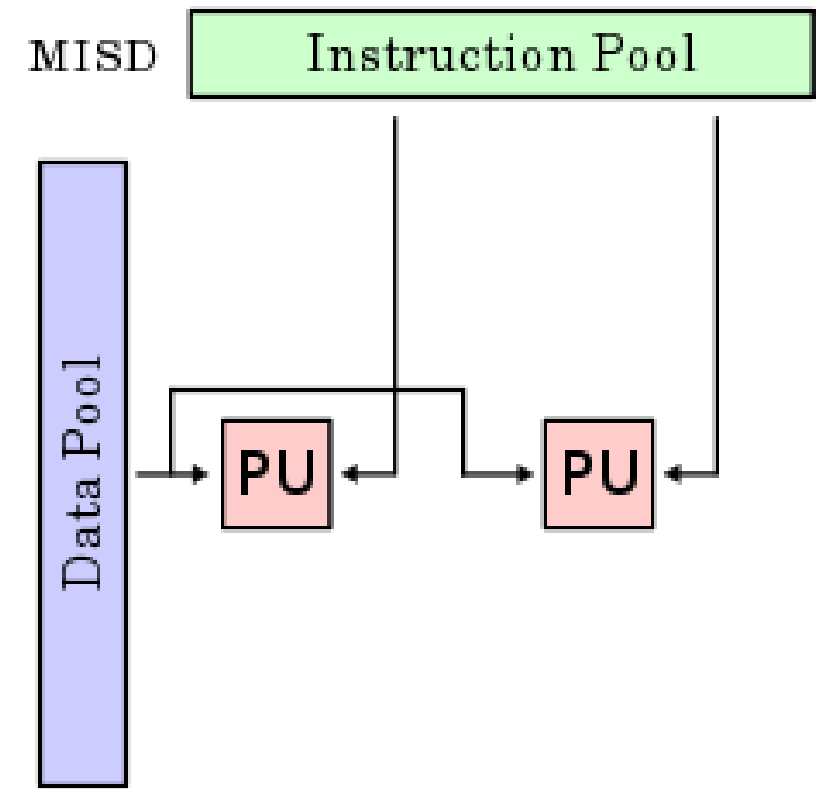
Single instruction stream, multiple data streams

- ❑ Throughput of the system can be increased by increasing the number of cores of the processor.
- ❑ Processing speed is higher than SISD architecture.
- ❑ There is complex communication between numbers of cores of processor.
- ❑ The cost is higher than SISD architecture.



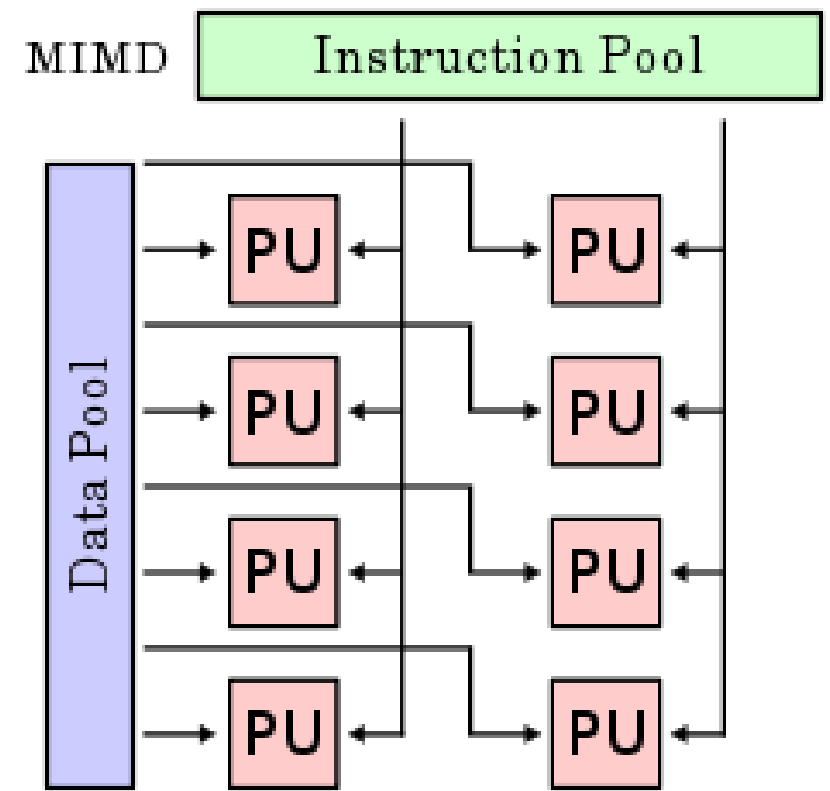
Multiple instruction streams, single data stream

- ❑ Multiple instructions operate on one data stream.
- ❑ Systems with MISD stream have number of processing units performing different operations by executing different instructions on the same data set.



Multiple instruction streams, multiple data streams

- ❑ Multiple autonomous processors simultaneously executing different instructions on different data.
- ❑ MIMD architectures include multi-core superscalar processors, and distributed systems, using either one shared memory space or a distributed memory space.



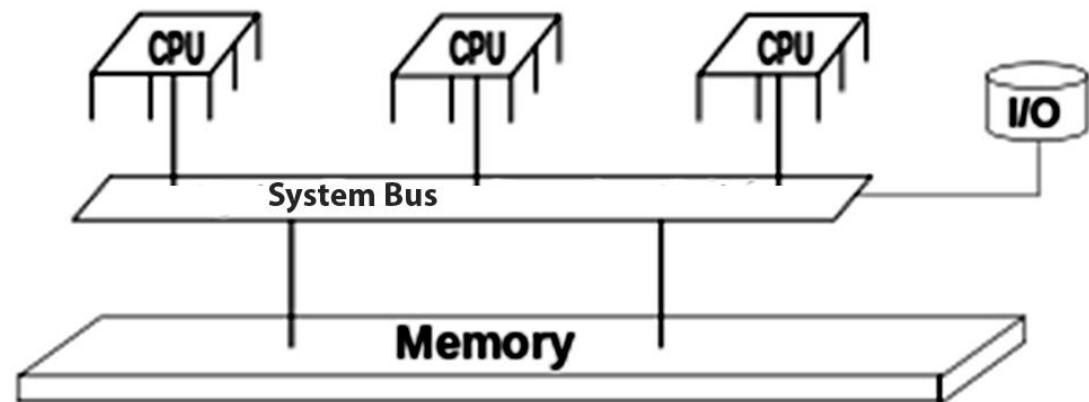
Summary

16

Classification	Instruction Streams	Data Streams	Typical Examples	Processing Type
SISD	1	1	Intel 8086, single-core CPUs	Sequential
SIMD	1	Many	GPUs, Vector processors, Intel AVX	Data parallelism
MISD	Many	1	Fault-tolerant systems (spacecraft control)	Redundant/fault-tolerant
MIMD	Many	Many	Multi-core CPUs, clusters, supercomputers	Task parallelism

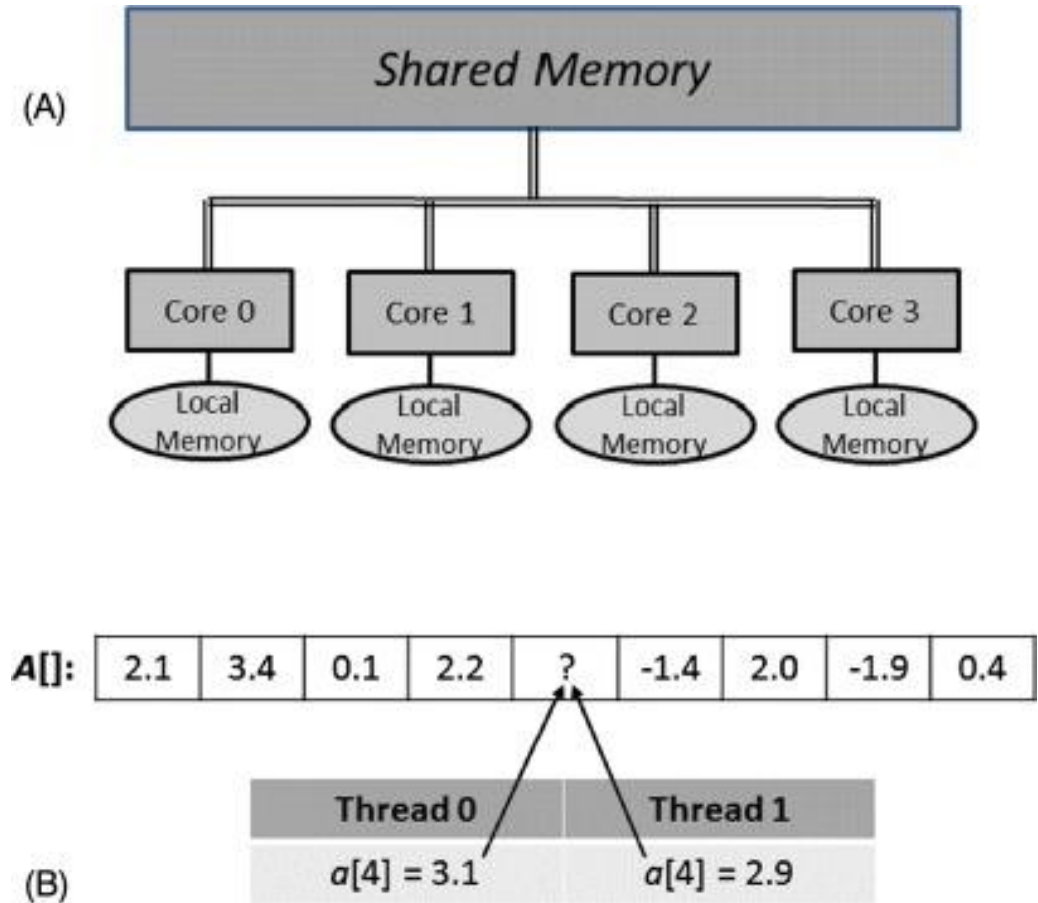
Multiprocessors (Shared Memory)

- ❑ Shared memory is memory that simultaneously accessed by multiple programs with an intent to provide communication among them or avoid redundant copies.
- ❑ Shared memory is an efficient means of passing data between programs.
- ❑ Depending on context, programs may run on a single processor or on multiple separate processors.



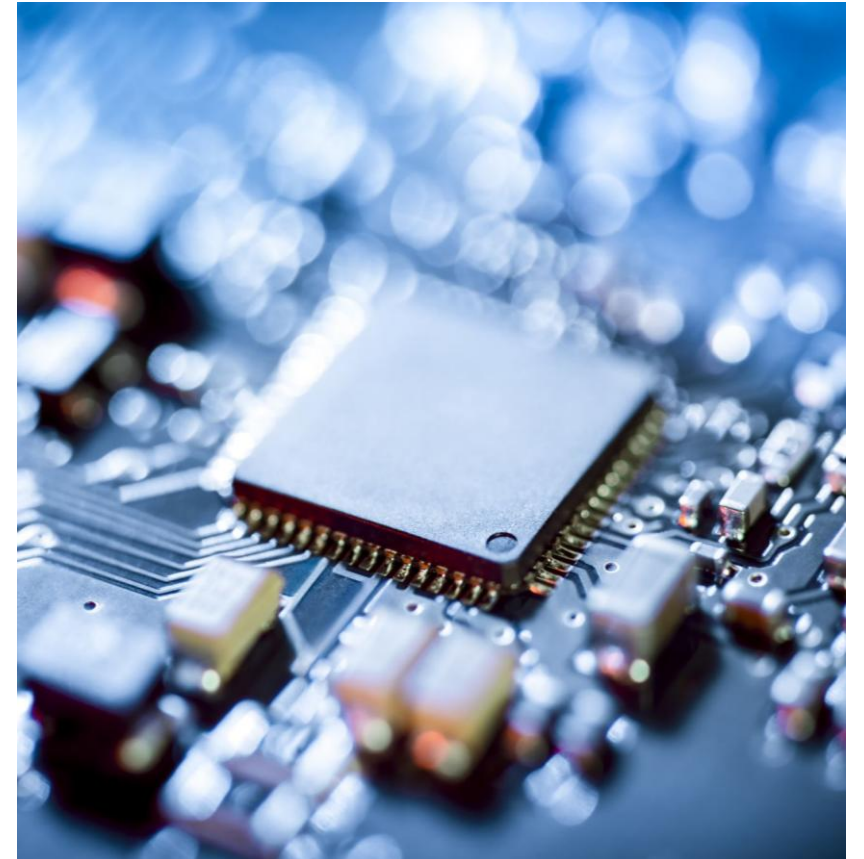
Shared Memory

- ❑ Shared memory concept is not only existed at hardware.
- ❑ It is rather a joint venture of Hardware and Software.



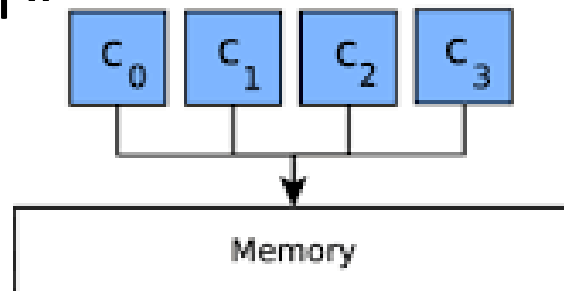
Shared Memory (Hardware Level)

- ❑ In computer hardware, shared memory refers to a (typically large) block of **Random Access Memory (RAM)** that can be accessed by **several different Central Processing Units (CPUs)** in a **multiprocessor computer system**.
- ❑ In computer architecture, and especially in Multiprocessors systems, memory access models play a critical role that determines **performance**, **scalability**, and **efficiency** of the system.
- ❑ The two shared-memory models most frequently used are **UMA** and **NUMA**.

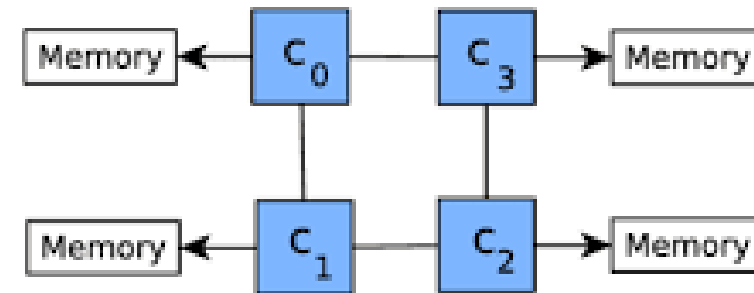


Shared Memory (Hardware Level)

- ❑ **Uniform memory Access (UMA):** all the processors share the physical memory uniformly (e.g. access time is same etc.);
- ❑ **Non-uniform Memory Access (NUMA):** memory access time depends on the memory location relative to a processor;
- ❑ **Cache-only Memory Architecture (COMA):** the local memories for the processors at each node is used as cache instead of as actual main memory



UMA



NUMA

Uniform Memory Access (UMA)

- ▣ In UMA, where Single memory controller is used.
- ▣ Uniform Memory Access is slower than non-uniform Memory Access.
- ▣ In Uniform Memory Access, bandwidth is restricted or limited rather than non-uniform memory access.
- ▣ There are 3 types of buses used in UMA which are:
 - Single, Multiple and Crossbar.
 - It is applicable for general purpose applications and time-sharing applications.

Advantages of Uniform Memory Access (UMA)

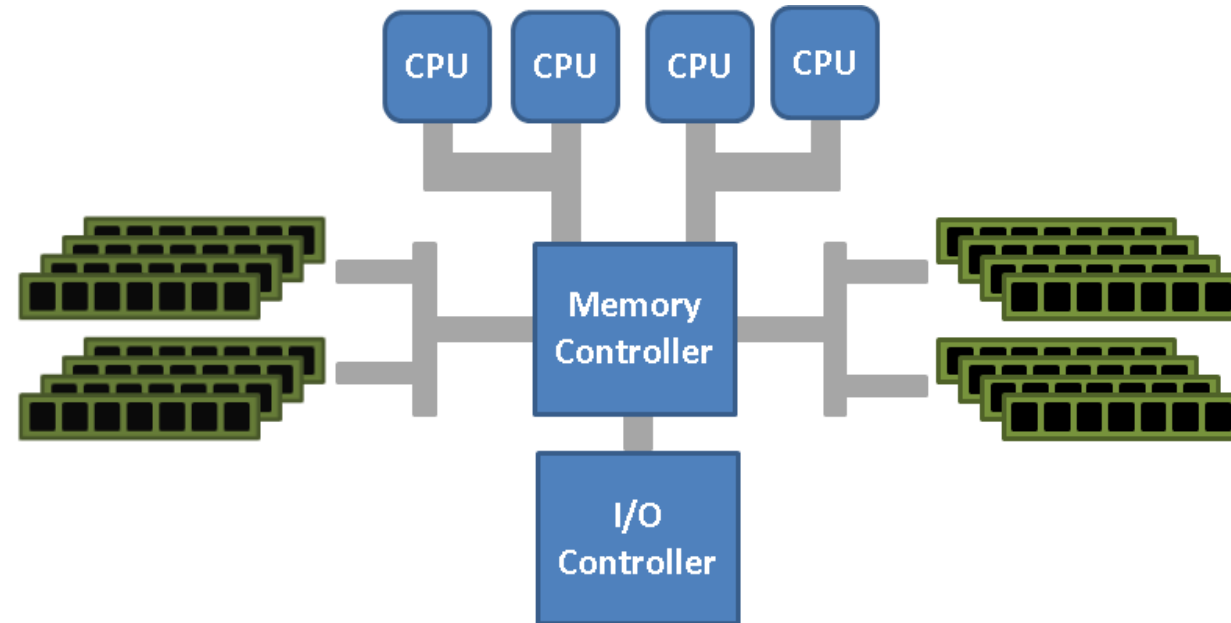
- **Easy to Implement:** UMA architecture is relatively **easy to implement**, as all processors or cores have equal access to the memory pool. This makes it an ideal choice **for small-scale systems**, such as desktop computers or low-end servers.
- **Low Latency:** Since all memory locations have equal access times, UMA provides **low latency**, which ensures that processors or cores can access memory quickly and efficiently. This makes UMA ideal for **high-performance computing** applications that require fast memory access.
- **Low Cost:** UMA architecture is relatively inexpensive to implement, as it requires only a single shared memory bus to connect all processors or cores to the memory pool. This makes it an ideal choice for low-cost computing systems.

Disadvantages of Uniform Memory Access (UMA)

- ❑ **Limited Scalability:** UMA architecture is not scalable beyond a certain point, as adding more processors or cores to the system can cause **contention** for the memory bus. This can result in reduced system performance as processors or cores have to wait for memory access.
- ❑ **Limited Bandwidth:** UMA architecture provides limited bandwidth, as all processors or cores share a single memory bus. This can result in reduced performance for memory-intensive applications.
- ❑ **Limited Memory Capacity:** UMA architecture provides limited memory capacity, as all processors or cores share a single memory pool. This can **limit the amount of memory** available to each processor or core, which can affect system performance.

Non-Uniform Memory Access (NUMA)

- In NUMA, where different memory controller is used.
- Non-uniform Memory Access is faster than uniform Memory Access.
- Non-uniform Memory Access is applicable for real-time applications and time-critical applications.



Advantages of a Non-Uniform Memory Access (NUMA)

- ❑ **Improved performance:** By providing each processor with its own local memory, NUMA can reduce memory access times and improve overall system performance.
- ❑ **Scalability:** NUMA systems are highly scalable and can handle large workloads by adding additional processors and memory nodes.
- ❑ **Reduced memory contention:** NUMA can help reduce memory contention by allowing each processor to access its own local memory, reducing the need for multiple processors to access the same memory location.

Disadvantages of Non-Uniform Memory Access (NUMA)

- ❑ **Complexity:** NUMA systems can be complex to design and implement, as they require specialized hardware and software to manage memory access.
- ❑ **Higher cost:** NUMA systems can be more expensive than UMA systems due to the additional hardware and software required.
- ❑ **Performance variability:** In some cases, the performance of a NUMA system may be lower than that of a UMA system, especially if the workload requires frequent access to shared memory.

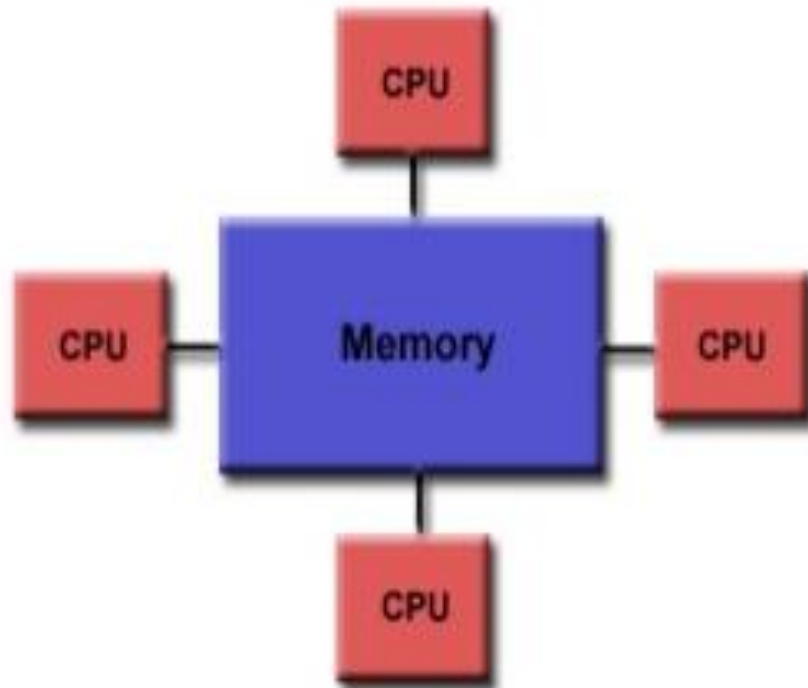
Similarities Between UMA And NUMA

- ❑ Both UMA and NUMA architectures are used in symmetric multiprocessing (SMP) systems where multiple processors share a common memory pool.
- ❑ Both UMA and NUMA architectures require memory management to ensure that all processors can access the memory they need.
- ❑ Both UMA and NUMA architectures require the use of cache memory to improve memory access times and reduce latency.
- ❑ Both UMA and NUMA architectures can be used in a wide range of applications, including scientific computing, database management, and high-performance computing.
- ❑ Both UMA and NUMA architectures can benefit from using high-speed interconnects to improve memory access times and reduce latency.

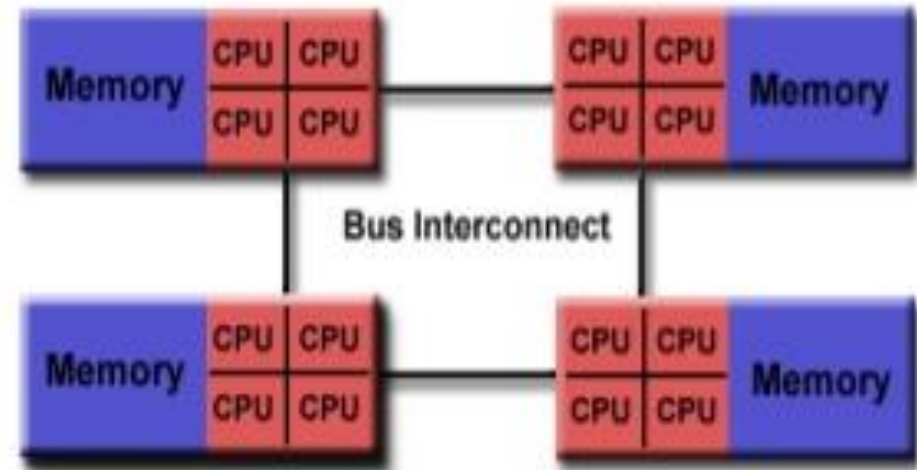
Difference between UMA and NUMA

S. No.	UMA	NUMA
1.	UMA stands for Uniform Memory Access.	NUMA stands for Non-uniform Memory Access.
2.	In Uniform Memory Access, Single memory controller is used.	In Non-uniform Memory Access, Different memory controller is used.
3.	Uniform Memory Access is slower than non-uniform Memory Access.	Non-uniform Memory Access is faster than uniform Memory Access.
4.	Uniform Memory Access has limited bandwidth.	Non-uniform Memory Access has more bandwidth than uniform Memory Access.
5.	Uniform Memory Access is applicable for general purpose applications and time-sharing applications.	Non-uniform Memory Access is applicable for real-time applications and time-critical applications.
6.	In uniform Memory Access, memory access time is balanced or equal.	In non-uniform Memory Access, memory access time is not equal.
7.	There are 3 types of buses used in uniform Memory Access which are: Single, Multiple and Crossbar.	While in non-uniform Memory Access, There are 2 types of buses used which are: Tree and hierarchical.
8.	Examples of UMA architecture- • Sun Starfire Servers • Compaq alpha server • HP v series	Examples NUMA architecture- • Cray • TC-2000 • BBN and others.

Shared Memory (Hardware Level)



Uniform Memory
Access (UMA)



Non-Uniform Memory
Access(NUMA)

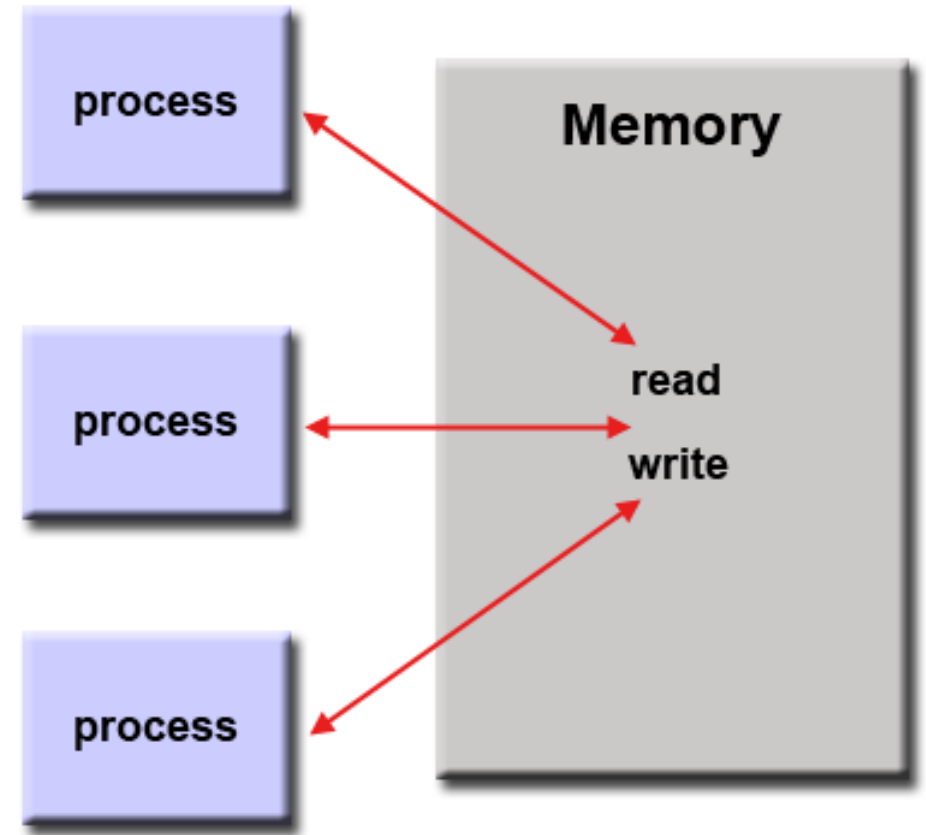
Shared Memory (Hardware Level)

- **An Advantage**: A shared memory system is relatively **easy to program** since all processors share a **single view of data** and the **communication between processors can be as fast as memory accesses to a same location**.
- **A Challenge**: The issue with shared memory systems is that **many CPUs need fast access to memory and will likely cache memory**, which **has two complications**:
 - ▣ **Access Time Degradation**: when **several processors try to access same memory location** it causes **contention (conflict)**.
 - ▣ **Lack of data Coherence**

Shared Memory (Hardware Level)

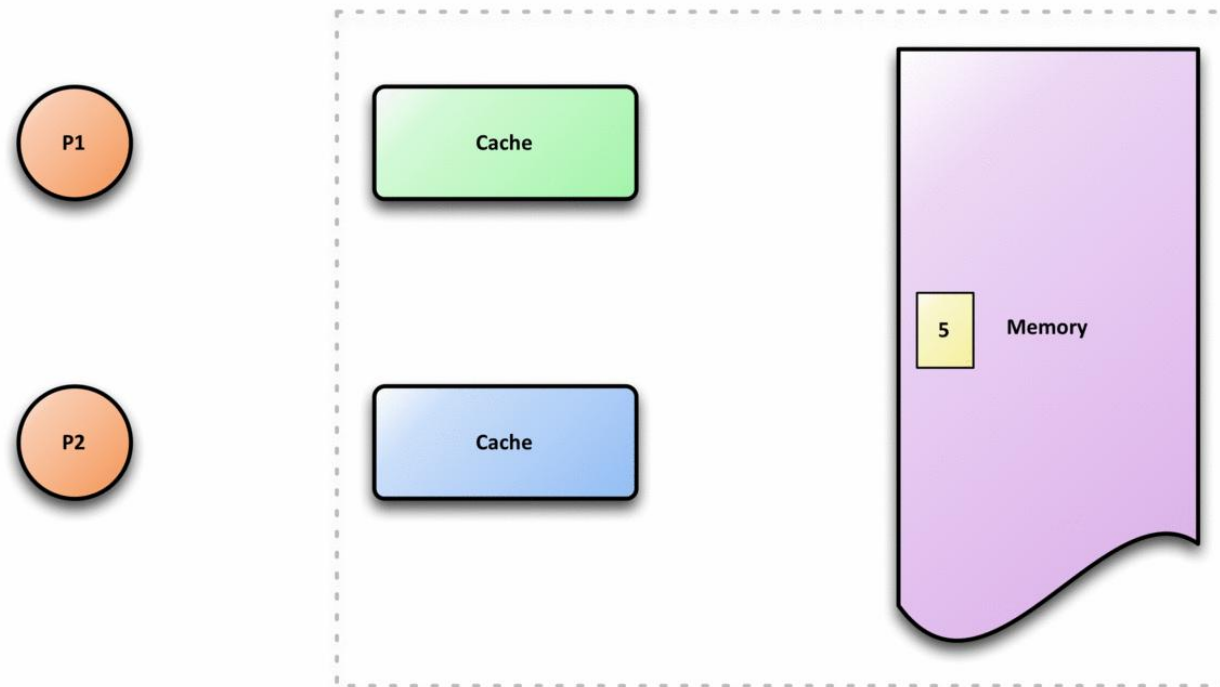
□ Access Time Degradation:

Various mechanisms can be used to aid this, including locks, semaphores, mutexes.



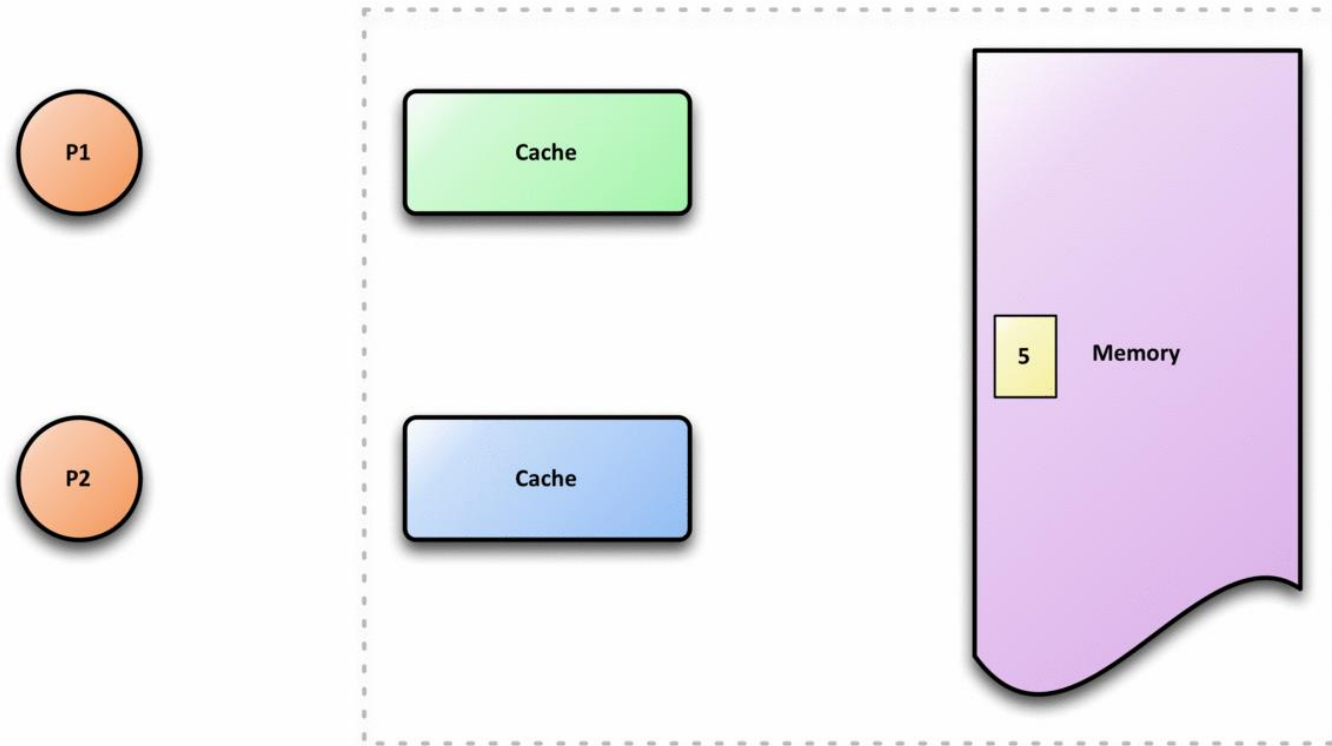
Lack of data Coherence

- Whenever **one cache is updated with information that may be used by other processors**, the **change needs to be reflected to the other processors**, otherwise the different processors will be working with **incoherent data**.



Incoherent caches: The caches have different values of a single address location.

Coherent caches



Coherent caches: The data in all the caches' copies is the same.

Shared Memory (Software Level)

- At software level, shared memory can be managed by either:
 1. A method of inter-process communication (IPC), i.e. **a way of exchanging data between programs running at the same time.**
 2. A method of conserving memory space **by directing accesses to what would ordinarily be copies of a piece of data to a single instance instead, by using virtual memory mappings or **with explicit support of the program in question.****

Shared Memory (Software Level)

- Since both processes **can access the shared memory area like regular working memory**, this is a very **fast way of communication**.
- On the other hand, **it is less scalable**, and **care must be taken to avoid issues if processes sharing memory are running on separate CPUs and the underlying architecture is not cache coherent**.

Shared Memory as Inter-Process Communication (IPC)

36

- **Shared memory** is used as a **channel for data exchange** between multiple processes running concurrently.
- Instead of sending messages or writing files, processes directly read/write to a **common memory area**.
- **Example (Linux):**
 - ▣ Process A (a producer) writes data to a shared memory region.
 - ▣ Process B (a consumer) reads that same data.Both processes use system calls like:

Shared Memory as Inter-Process Communication (IPC)

37

□ **Example scenario:**

- ▣ Imagine two programs — one collects sensor data, and another displays it on a dashboard:
- ▣ The **sensor program** stores the latest readings in shared memory.
- ▣ The **display program** reads directly from that memory to update the screen instantly.
- ▣ This avoids the overhead of copying or sending data through files or sockets.

□ **Real-life example:**

- ▣ **Databases** like PostgreSQL use shared memory for communication between different server processes.
- ▣ **Operating Systems** use it for exchanging kernel-user data efficiently.

Shared Memory for Conserving Memory Space

38

- Sometimes, multiple programs or processes use **the same data (e.g., libraries, files, or constants)**.
- Instead of loading **separate copies** of that data for each process, the OS arranges memory so that **they all refer to the same instance** — saving memory space.
- This is achieved using:
 - ▣ **Virtual memory mappings**
 - ▣ **Memory-mapped files**
 - ▣ **Shared libraries**

Shared Memory for Conserving Memory Space

39

- ❑ **Example (Virtual Memory Mapping):**
 - ▣ Suppose you open the same image file in two photo viewer programs:
 - ▣ Normally, each would load its own copy into memory.
 - ▣ But the OS can map both programs' address spaces to the **same physical memory pages** using **memory mapping** (`mmap()` in Linux).
 - ▣ Now both programs **see** their own copy of the image, but physically it's **one shared copy** in RAM.
- ❑ **Real-life examples:**
 - ▣ **Shared libraries (.dll or .so):** The same library code (e.g., `libc.so`) is mapped into memory once and shared by all programs that use it.
 - ▣ **Copy-on-Write (COW):** When you open multiple instances of a program, they share the same memory pages until one modifies them — then the OS creates a copy.

IPC vs Memory Conservation

40

Purpose	Description	Example	Benefit
IPC (Communication)	Processes exchange data through a common shared memory area.	Two processes — one writes sensor data; another reads it to display.	Fastest form of inter-process data sharing.
Memory Conservation	Processes share common data or code via virtual memory mappings.	Multiple apps using the same shared library or file via mmap().	Saves memory by avoiding duplicate copies.

Thanks

41

“

**All dreams are
within reach.**

All you have to do is
**keep moving
towards them.**

—*Viola Davis*

Prevention